SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3) Assessment

Tool Weightage: 40% Duration: 1 Hour Total Marks: 30

ANNEXURE A — DRY RUN EVALUATION

Code of Conduct:

I __Prabhavathi R (192321101)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANNEXURE A — DRY RUN EVALUATION

1. Question 1 — Breadth-First Search (BFS)

Given graph: A is the root; B and C are its children; D and E are children of B; F and G are children of C. We visit nodes level by level from left to right.

BFS traversal: A, B, C, D, E, F, G

Queue rule: Remove the first node, visit it, and add its unvisited children to the end of the queue.

Iteration	Queue before	Visited node	Queue after
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

Answers:

1. Traversal order: A → B → C → D → E → F → G.
2. Queue contents after each iteration are shown in the table. The queue follows FIFO (First In, First Out).
3. BFS visits all nodes at depth 0, then depth 1, then depth 2.
4. Time complexity: O(V + E), where V is the number of vertices and E is the number of edges.
5. Space complexity: O(V), because the queue may store many nodes.

2. Question 2 — Depth-First Search (DFS)

DFS explores one branch as far as possible before backtracking. Assuming left-to-right order, it visits A, then B, then D, backtracks to B, visits E, then C, F and G.

DFS traversal: A → B → D → E → C → F → G

Step	Action	Stack (top at right)	Visited order
1	Start at A	[A]	A
2	Go to B	[A, B]	A, B
3	Go to D	[A, B, D]	A, B, D
4	Backtrack; visit E	[A, B, E]	A, B, D, E
5	Backtrack; go to C	[A, C]	A, B, D, E, C
6	Go to F	[A, C, F]	A, B, D, E, C, F
7	Visit G	[A, C, G]	A, B, D, E, C, F, G

Explanation: DFS uses a stack (Last In, First Out) or recursion. It explores D before E because D is the left child of B. After finishing B's branch, it moves to C.

Complexity: Time O(V + E); space O(V) in the worst case.

3. Question 3 — 8-Puzzle Using BFS

Initial state:

1	**3**	**6**
5	**0**	**2**
4	**7**	**8**

Goal state:

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

BFS explores states in order of the number of moves from the initial state. The blank (0) can move up, down, left or right when the move stays inside the 3×3 board. Duplicate states are not added again.

Important: The supplied initial state is not solvable for the supplied goal state. For a 3×3 puzzle, the inversion parity must match. The initial sequence ignoring 0 is 1, 3, 6, 5, 2, 4, 7, 8 and has 7 inversions (odd), while the goal has 0 inversions (even). Therefore, BFS cannot reach the goal from this initial state.

Dry-run table (first five expansions):

Iteration	Current state (rows)	Queue before expansion	Queue after expansion
1	136 / 502 / 478	[S0]	[S1, S2, S3]
2	136 / 502 / 478 (S1)	[S1, S2, S3]	[S2, S3, new children of S1]
3	136 / 502 / 478 (S2)	[S2, S3, ...]	[S3, ..., new children of S2]
4	136 / 502 / 478 (S3)	[S3, ...]	[..., new children of S3]
5	Next queued state	[...]	[...]

Because the question's initial state is unsolvable, a fully valid goal-reaching BFS trace and solution path do not exist. The table format above illustrates the queue procedure, but exact state-by-state queue contents depend on the order used to generate legal moves. Do not report a goal path for this initial state.

Answers to the questions:

1. The expanded node in each iteration is the state at the front of the FIFO queue.
2. The total number of generated states before reaching the goal cannot be given: the goal is unreachable fromthis initial state. With duplicate checking, BFS will eventually exhaust all states reachable in the same parity class.
3. No solution path exists for the stated initial and goal states because the puzzle is unsolvable.
4. BFS finds a shortest solution in an unweighted state space when a solution exists because it expands states inincreasing order of depth (number of moves). It cannot find a solution when the goal is unreachable.

Note: In the provided graph, the child nodes are interpreted from left to right as D, E under B and F, G under C.